❖ Recursion in C++

- Recursion is a process in which a function calls itself again and again until a stopping condition (base condition) is reached.

OR

- Recursion is a technique where a function calls itself to solve a problem.
- Syntax

```
return_type function(parameters)
{
   if(base_condition)
   {
      return;
   }

   function(arguments);   // Recursive call
}
```

Example

Q. Write a program in C++ to print 10 to 0 series using recursion.

```cpp
#include <iostream>
#include<conio.h>
using namespace std;

void hira(int x)
{
    if(x>=0)
    {
        cout << x << "  " ;
        hira(x-1); // function call (it self)

    }
}
int main()
{
    int a=10;
    hira(a); // function call
    getch();
    return 0;

}
```



A recursive function mainly has two important parts:

1. **Base Condition**
2. **Recursive Call**

1. Base Condition

- The **base condition** is the condition that stops the recursion.
- Without a base condition, the function may call itself indefinitely (infinite loop).
- Example :

```
void Hira(int n)
{
   if(n == 0)
      return;

   Hira(n - 1); // recursive call
}
```

Here :

```
if(n == 0)
  return;
```

(is the **base condition**.)

2. Recursive Call

- When a function calls itself, it is called a **recursive call**.
- Above syntax –
- Here :

```
Hira(n-1); // recursive call
```

Q. Find factorial using Recursion

```
using namespace std;
int hira(int a)
    {
    if (a == 0)  // base case
        return 1;
    else
        return a * hira(a - 1);  // recursive call
    }
int main()
{
    int a;
  cout << "enter number = "; cin >> a;
  hira(a);
    cout << "Factorial of " << a << " is " << hira(a);
    getch();
}
```

```
J:\c-c++\part-3\5.exe
enter number = 5
Factorial of 5 is 120
```

❖ Bounded & Unbonded Recursion

Bounded Recursion

- Bounded recursion means recursion that has a fixed or limited number of recursive calls, or a clearly defined stopping condition.
- Example :

```
void hira (int a)
            {
                If(a==0)
                    Return;
                Cout << a << “  “ ;
                hira(a-1);
            }
int main()
{
hira(5);
return 0;
```

}
Output
5 4 3 2 1

Unbounded Recursion

- Unbounded recursion means recursion that has no effective stopping condition and may continue until the program runs out of stack space.
- Example :

```
Void hira()
{
    hira();
}
```

Here, the function calls itself repeatedly without a base case, potentially causing a stack overflow.

❖ Stack Building & Stack Unwinding

Stack Building

- Stack building is the process of adding function call information to the call stack when a function calls itself recursively or calls another function.
- Example : When hira(3) calls hira(2), which calls hira(1), new function-call information is added to the call stack.

Stack Unwinding

- Stack unwinding is the process in which function calls return one by one, and their information is removed from the call stack.
- Example : After the base condition is reached in recursion, the function calls return in reverse order, such as hira(1), hira(2) and hira(3).

Recursion based question

Q1. Write a program in C++ to print Fibonacci series.

Q2. Write a program in C++ to print Sum of first N natural numbers.

Q3. Write a program in C++ to print reverse any number.

Q4. Write a program in C++ to print sum of digit.

Note :- Q. In C++, Can main() call itself ? ➔ Yes , it is possible, but not recommended.

(Technically allowed, but practical avoid this concept)

❖ Inline Function

- An inline function is a function in which the compiler is requested to replace the function call with the actual function code at the place where the function is called.
- It reduces function call overhead and makes execution faster for small functions.
- It is declared using the inline keyword.
- inline is a request to the compiler, not a guarantee.
- Syntax :

```
inline return_type function_name(parameters)
{
   // statements
}
```

- E.g. – **Find square using inline function**

```
#include<iostream>
#include<conio.h>
using namespace std;
inline int hira(int a) //inline function
{
    return a*a;
}
int main()
{
    int x=5;
    cout << "Square = " << hira(x);
    //compiler internally replace cout<<"square=" << x*x;
    getch();
    return 0;
}
```

```
C:\Users\hira\Desktop\hira.exe
Square = 25
```

❖ Advantages

- Reduces function-call overhead.
- Can improve performance for small, frequently called functions.
- Useful for simple functions such as getters, setters, and small calculations.

❖ Disadvantages

- Excessive inline expansion can increase program size.
- Not suitable for every function.
- Large/complex functions are generally poor candidates.

❖ Difference b/w Normal function & inline function

Normal Function	Inline Function
Normal function call takes place	Compiler may expand code at call site
Function-call overhead may occur	Call overhead can be reduced
No inline keyword required	inline keyword can be used
Suitable for all types of functions	Mainly useful for small functions

❖ Dereferencing

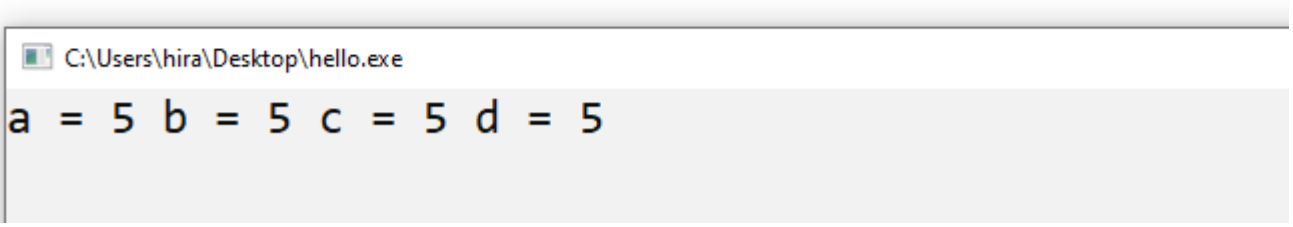
- Dereferencing in C++ is the process of accessing the value stored at the memory address pointed to by a pointer using the *(asterisk) operator.
- Example :

```
int a= 5;
int *b=&a;
cout << b ; // 5
```

Here - a stores the value 5.
&a gives the address of a.
stores the address of a.
*b accesses the value at the address which is 5.

```cpp
#include <iostream>
#include<conio.h>
using namespace std;

int main()
{
    int a=5, *b,**c,***d;
    b=&a;
    c=&b;
    d=&c;
    cout << "a = " << a << " b = " << *b << " c = " << **c<< " d = " << ***d;
    getch();
}
```



❖ Mix questions

1.	int a = 10;
	int *p = &a;
	*p = *p + 5;
	cout << a;

2.	int a = 5, b = 10;
	int *p = &a;
	p = &b;
	*p = 20;
	cout << a << " " << b;

3.	int a = 10;
	int *p = &a;
	int **q = &p;
	**q = 25;
	cout << a;

4.	int a = 4, b = 7;
	int *p = &a;
	int *q = &b;
	*p = *q + 3;
	cout << a << " " << b;

5.	int a = 10;
	int *p = &a;
	int *q = p;
	*q = *p + 5;

	*p = *q + 2; cout << a;

6.	int a = 10;
	int &r = a;
	r = 20;
	cout << a;

7.	int a = 5, b = 8;
	int &r = a;
	r = b;
	cout << a << " " << b;

8.	int x = 10;
	int &y = x;
	int &z = y;
	z = z + 5;
	cout << x << " " << y << " " << z;

9.	int a = 3, b = 6;
	int &r = a;
	r = r * b;
	b = r + b;
	cout << a << " " << b;

10	int a = 10, b = 20;
	int &r = a;
	r = b + 5;
	b = r + 10;
	cout << a << " " << b;

11	int a = 5, b = 10;
	cout << (a > 10 && ++b > 10);
	cout << " " << b;

12	int a = 5, b = 10;
	cout << (a < 10 || ++b > 10);
	cout << " " << b;

13	int x = 0, y = 5; if (x != 0 && y++ > 3) cout << "Yes"; else

	cout << "No"; cout << " " << y;
14	int a = 0, b = 5; if (a == 0 || ++b > 5) cout << b; else cout << a;

15	int a = 2, b = 3;
	int result = (a++ > 2 && ++b > 3) || ++a > 3;
	cout << result << " " << a << " " << b;

16	int a = 5;
	int *p = &a;
	int &r = *p;
	r = r + 10;
	cout << *p << " " << a;

17	int a = 4, b = 8;
	int *p = &a;
	int &r = b;
	*p = r + 2;
	r = *p + 3;
	cout << a << " " << b;

18	int a = 5;
	int *p = &a;
	int &r = a;
	if (*p > 5 && ++r > 5)
	cout << a;
	else
	cout << r;

19	int a = 2, b = 4;
	int *p = &a;
	int &r = b;
	cout << ((*p == 2 || ++r > 5) && b > 4);
	cout << " " << a << " " << b;

20	int a = 3, b = 6; int *p = &a;

	int &r = b;
	*p = r / 2;
	cout << (a > 2 && b++ > 5);
	cout << " " << a << " " << b;

Q1. 15
Q2. 5 20
Q3. 25
Q4. 10 7
Q5. 17
Q6. 20
Q7. 8 8
Q8. 15 15 15
Q9. 18 24
Q10. 25 35
Q11. 0 10
Q12. 1 10
Q13. No 5
Q14. 5
Q15. 1 4 3
Q16. 15 15
Q17. 10 13
Q18. 5
Q19. 1 2 4
Q20. 1 3 7

Error based questions

1.	int a = 10;
	int *p;
	*p = a;
	cout << *p;

2.	int a = 5;
	int *p = &a;
	int *q = &p;
	cout << **q;

3.	int a = 5; if (a > 10 || ++a > 5) cout << a;

Answer

1. p is an uninitialized pointer. It must point to a valid variable before dereferencing it.
2. &p has type int**, but q is declared as int*.
3. No error

❖ Advantages and Disadvantages of Functions in C++

Advantages of Functions

1. Code Reusability: A function can be used multiple times without writing the same code again.
2. Modularity: A large program can be divided into smaller, manageable parts.
3. Easy Debugging: Errors can be identified and corrected easily in individual functions.
4. Easy Maintenance: Changes can be made to a function without modifying the entire program.
5. Improves Readability: Functions make the program easier to understand.
6. Reduces Code Duplication: The same code does not need to be written repeatedly.
7. Teamwork: Different programmers can work on different functions of the same program.

Disadvantages of Functions

1. Function Call Overhead: Calling a function may require extra execution time and memory.
2. Memory Usage: Function calls require memory for parameters, local variables, and return addresses.
3. Complexity: Too many small functions can make a program difficult to follow.

4. Debugging Difficulty: When many functions call one another, tracing the flow of execution can become difficult.
5. Parameter Passing: Passing incorrect arguments or using incorrect parameter types can cause errors.
6. Recursion Overhead: Recursive functions may consume a large amount of stack memory if not designed properly.

==========HIRA KUMAR==========